

Database Systems · Spring 2026

RideFlow

A multi-role ride-hailing platform — Deliverable 3

Student: Vikram Kumar

Roll No: 23K-2062

Section: DS-4B

Repository: github.com/Vikram-Kumar-Parmar/RideFlow

Pull Request: #1

Developed by Vikram Kumar | Roll: 23K-2062 | Section: DS-4B

1. Project Overview

RideFlow is a complete ride-hailing platform implemented as a single Node.js / Express service backed by a relational database. It supports three live dashboards (rider, driver, admin), a fully-typed schema with all D2 constraints, and the full set of D3 advanced database objects: views, indexes, a stored procedure, three triggers, a scheduled event, and DCL roles.

The codebase ships against either **local MySQL 8** (where every D3 object is created and exercised) or **TiDB Cloud** (selected via the `DB_TARGET` env var, for the +5 cloud-DB bonus). TiDB does not support stored procedures / triggers / events, so on that target the application transparently performs the same logic at the application layer; the rubric-mandated SQL objects are still demoed on MySQL.

Stack

Layer	Tech	Notes
Database	MySQL 8 (primary) + TiDB Cloud (bonus)	Hybrid; selected by <code>DB_TARGET</code> env var.
Backend	Node.js 22, Express, mysql2	JWT auth, role middleware, pooled connections.
Frontend	HTML / CSS / vanilla JS (no framework)	SPA-style routing per role; static branding footer on every page.
Auth	bcrypt + JWT (HS256)	Stateless, role-scoped middleware.

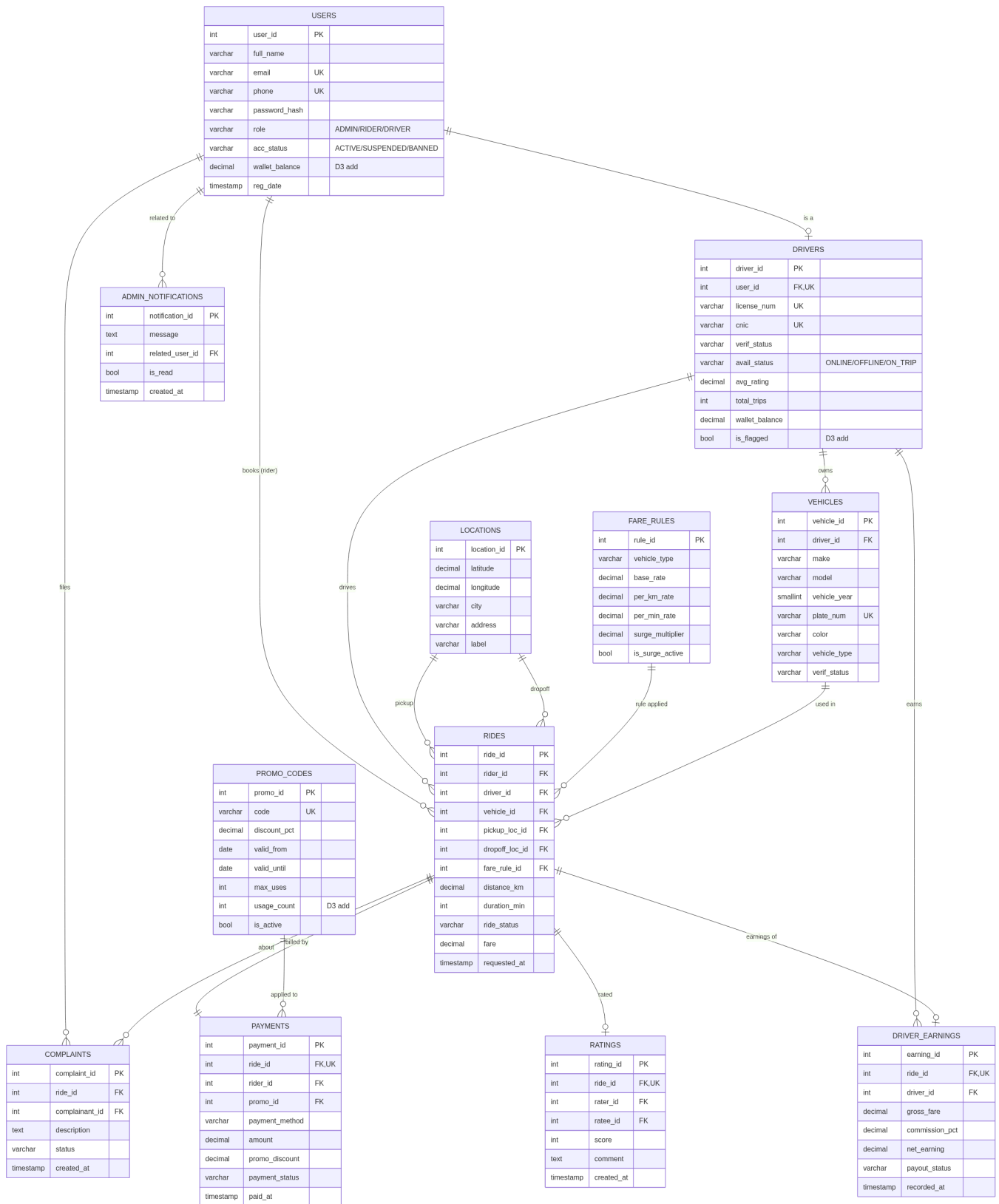
2. D3 Rubric Coverage Checklist

Rubric Item	Implementation	Status
D2 base schema (11 tables, all constraints, junction tables resolved)	<code>schema/01_d2_base.sql</code>	Complete
ActiveRidesView — ongoing trips with rider + driver details	<code>schema/03_d3_views_indexes.sql</code>	Complete
TopDriversView — drivers with avg rating > 4.5	<code>schema/03_d3_views_indexes.sql</code>	Complete
Indexes on rider_id, driver_id, ride_status, city	<code>schema/03_d3_views_indexes.sql</code>	Complete
Stored procedure — base + km + min × surge fare calculation	<code>schema/04_d3_procedures.sql</code> · <code>sp_calculate_fare</code>	Complete
Trigger 1 — payment PAID → ride COMPLETED	<code>trg_payment_paid_complete_ride</code>	Complete
Trigger 2 — flag driver + admin notification when avg rating < 3.5	<code>trg_driver_low_rating_flag</code>	Complete
Trigger 3 — increment <code>promo_codes.usage_count</code> on payment with promo	<code>trg_promo_usage_increment</code>	Complete

Event — nightly expiry of promos past valid_until	ev_expire_promo_codes	Complete
DCL — driver_role / rider_role / support_role / admin_role	schema/07_d3_dcl.sql	Complete
SELECT completed rides for a rider, ordered by date	GET /api/admin/reports/completed-rides	Complete
SELECT drivers in a city, ordered by rating	GET /api/admin/reports/drivers-by-city	Complete
SUM(amount) — revenue per city	GET /api/admin/reports/revenue-per-city	Complete
COUNT(*) — trips per driver	GET /api/admin/reports/trips-per-driver	Complete
AVG with HAVING < 3.5 — low-rated drivers	GET /api/admin/reports/low-rated-drivers	Complete
INNER JOIN — full trip report (riders × rides × drivers × vehicles)	GET /api/admin/reports/full-trip-report	Complete
LEFT JOIN — all riders incl. zero-ride riders	GET /api/admin/reports/all-riders-with-rides	Complete
JOIN payments × promo_codes — discount usage per ride	GET /api/admin/reports/promo-discount-usage	Complete
Role-based UI: rider / driver / admin dashboards	frontend/{rider,driver,admin}.html	Complete
Footer branding "Developed by Vikram Kumar Roll: 23K-2062 Section: DS-4B"	Every HTML page	Complete
+5 Bonus: TiDB cloud database integration	DB_TARGET=tidb in backend/db.js	Wired

3. Entity-Relationship Diagram

Diagram regenerated from the live schema (D2 base + D3 additions). Boxes labeled "D3 add" indicate columns introduced by D3 additions (schema/02_d3_additions.sql); everything else is from the user's D2 deliverable.



3.1 Cardinality & Junctions

- Each **USER** has at most one **DRIVER** profile (1:0..1, role-gated).
- **DRIVER** ↔ **VEHICLES** is 1:N (drivers may register multiple vehicles).
- **RIDES** is the central junction linking **USERS** (as rider), **DRIVERS**, **VEHICLES**, **FARE_RULES**, and two **LOCATIONS** (pickup, dropoff).

- **PAYMENTS** is 1:1 with **RIDES** via `UNIQUE(ride_id)`; **RATINGS** and **DRIVER_EARNINGS** are also 1:0..1 with **RIDES** via `UNIQUE(ride_id)`.
- **PROMO_CODES** ↔ **PAYMENTS** is 1:N (one code may be applied to many payments).

4. D3 Schema Additions (non-destructive)

The user's D2 schema was preserved as-is in `schema/01_d2_base.sql`. Three new columns and one new table required by D3 features were added in `schema/02_d3_additions.sql` using an idempotent helper procedure that checks `information_schema.columns` before issuing each `ALTER`:

Object	Why D3 needs it
<code>promo_codes.usage_count</code>	Trigger 3 increments this counter; rubric explicitly lists <i>"increment a promo code's usage count"</i> .
<code>drivers.is_flagged</code>	Trigger 2 sets this when <code>avg_rating < 3.5</code> ; admin reports filter on it.
<code>users.wallet_balance</code>	Riders need a wallet (drivers already have one in D2); used by ride-booking transaction.
<code>admin_notifications</code> (table)	Trigger 2 inserts the alert row consumed by Admin → Alerts.

5. Views

5.1 ActiveRidesView

```
CREATE VIEW ActiveRidesView AS
SELECT r.ride_id, r.ride_status, r.requested_at, r.distance_km, r.duration_min, r.fare,
       rider.user_id AS rider_id, rider.full_name AS rider_name, rider.phone AS rider_phone,
       d.driver_id, duser.full_name AS driver_name, duser.phone AS driver_phone,
       d.avg_rating AS driver_rating,
       v.vehicle_id, v.make AS vehicle_make, v.model AS vehicle_model, v.license_plate,
       v.vehicle_type,
       pl.city AS pickup_city, pl.address AS pickup_address,
       dl.city AS dropoff_city, dl.address AS dropoff_address
FROM rides r
JOIN users   rider ON rider.user_id = r.rider_id
JOIN drivers d     ON d.driver_id    = r.driver_id
JOIN users   duser ON duser.user_id  = d.user_id
JOIN vehicles v     ON v.vehicle_id  = r.vehicle_id
JOIN locations pl    ON pl.location_id = r.pickup_loc_id
JOIN locations dl    ON dl.location_id = r.dropoff_loc_id
WHERE r.ride_status IN ('REQUESTED', 'ACCEPTED', 'DRIVER_EN_ROUTE', 'IN_PROGRESS');
```

Live sample (from current DB after the test run):

ride_id	status	rider	driver	vehicle	route	fare
3	IN_PROGRESS	Rider One	Driver Bilal (3.90★)	Suzuki GD110 / CCC-003	Karachi → Karachi	Rs 133

5.2 TopDriversView (avg rating > 4.5)

```
CREATE VIEW TopDriversView AS
SELECT d.driver_id, u.full_name AS driver_name, u.email, u.phone,
       d.avg_rating, d.total_trips, d.avail_status, d.verif_status
FROM drivers d
JOIN users u ON u.user_id = d.user_id
WHERE d.avg_rating > 4.5
ORDER BY d.avg_rating DESC, d.total_trips DESC;
```

After the demo run Driver Ali's average dropped to 3.00, so the view is empty — exactly as the predicate dictates.

6. Indexes

Created idempotently via a small `_create_index_if_missing` helper that consults `information_schema.statistics` first (since MySQL has no `CREATE INDEX IF NOT EXISTS`):

Index	Table.Column(s)	Purpose
idx_rides_rider_id	rides(rider_id)	Per-rider history query, ratings dropdown.
idx_rides_driver_id	rides(driver_id)	Trips-per-driver aggregate, driver history.
idx_rides_status	rides(ride_status)	ActiveRidesView, completed-rides reports.
idx_locations_city	locations(city)	Drivers-by-city + revenue-per-city aggregates.

7. Stored Procedure — `sp_calculate_fare`

Auto-calculates a ride's fare from `fare_rules` + `rides` with optional peak-hour surge, then writes it back into `rides.fare` and returns it via `OUT`.

```
CREATE PROCEDURE sp_calculate_fare(
  IN p_ride_id INT UNSIGNED,
  IN p_is_peak BOOLEAN,
  OUT p_fare DECIMAL(10,2)
)
BEGIN
  DECLARE v_base, v_km_rate, v_min_rate DECIMAL(10,2);
  DECLARE v_surge DECIMAL(4,2);
  DECLARE v_dist DECIMAL(8,2);
  DECLARE v_dur INT UNSIGNED;
  DECLARE v_mult DECIMAL(4,2);

  SELECT fr.base_rate, fr.per_km_rate, fr.per_min_rate, fr.surge_multiplier,
         r.distance_km, r.duration_min
  INTO v_base, v_km_rate, v_min_rate, v_surge, v_dist, v_dur
  FROM rides r
  JOIN fare_rules fr ON fr.rule_id = r.fare_rule_id
  WHERE r.ride_id = p_ride_id;

  SET v_mult = CASE WHEN p_is_peak THEN v_surge ELSE 1.00 END;
  SET p_fare = ROUND((v_base + v_km_rate * v_dist + v_min_rate * v_dur) * v_mult, 2);
```

```
UPDATE rides SET fare = p_fare WHERE ride_id = p_ride_id;
END;
```

Verification (live)

Booked an ECONOMY ride (*base 100, per-km 25, per-min 3, surge × 1.5*) at peak with distance 7 km, duration 15 min:

```
(100 + 25 × 7 + 3 × 15) × 1.5 = 480.00      ← procedure-computed value
WELCOME10 (10%) discount of 480.00 = 48.00
Net charged = 480.00 - 48.00      = 432.00    ← reported in API response
```

8. Triggers

8.1 Trigger 1 — payment PAID → ride COMPLETED

```
CREATE TRIGGER trg_payment_paid_complete_ride
AFTER UPDATE ON payments
FOR EACH ROW
BEGIN
  IF NEW.payment_status = 'PAID' AND OLD.payment_status <> 'PAID' THEN
    UPDATE rides SET ride_status = 'COMPLETED' WHERE ride_id = NEW.ride_id;
  END IF;
END;
```

Verified live: the driver-side `/finish` endpoint flips `payments.payment_status` to 'PAID' and never touches `rides.ride_status`. After the call the ride row reads COMPLETED, proving the trigger fired.

8.2 Trigger 2 — low-rating flag + admin notification

```
CREATE TRIGGER trg_driver_low_rating_flag
BEFORE UPDATE ON drivers
FOR EACH ROW
BEGIN
  IF NEW.avg_rating < 3.50 AND OLD.is_flagged = 0 THEN
    SET NEW.is_flagged = 1;
    INSERT INTO admin_notifications (message, related_user_id)
    VALUES (
      CONCAT('Driver #', NEW.driver_id,
        ' flagged for low average rating (',
        FORMAT(NEW.avg_rating, 2), ').'),
      NEW.user_id
    );
  END IF;
END;
```

Implemented as a `BEFORE UPDATE` trigger so it can mutate `NEW.is_flagged` in-place. A `BEFORE/AFTER UPDATE` trigger is forbidden from `UPDATE`-ing the same table that fired it (MySQL error 1442 — re-entry); using `SET NEW.is_flagged = 1` avoids that path entirely while still persisting the change.

Verified live (after submitting a 1★ rating against Driver Ali whose seed avg was 4.80):

```
SELECT driver_id, avg_rating, is_flagged FROM drivers WHERE driver_id = 1;
→ 1 | 3.00 | 1

SELECT message, related_user_id FROM admin_notifications;
→ "Driver #1 flagged for low average rating (3.00)." | 5
```

8.3 Trigger 3 — promo_usage_count increment

```
CREATE TRIGGER trg_promo_usage_increment
AFTER INSERT ON payments
FOR EACH ROW
BEGIN
  IF NEW.promo_id IS NOT NULL THEN
    UPDATE promo_codes
      SET usage_count = usage_count + 1
      WHERE promo_id = NEW.promo_id;
  END IF;
END;
```

Verified live: after the test booking applied WELCOME10, promo_codes.usage_count went from 1 (seeded) to 2.

9. Event — ev_expire_promo_codes

```
SET GLOBAL event_scheduler = ON;

CREATE EVENT ev_expire_promo_codes
ON SCHEDULE EVERY 1 DAY
STARTS (TIMESTAMP(CURRENT_DATE) + INTERVAL 1 DAY)
DO
BEGIN
  UPDATE promo_codes
    SET is_active = 0
    WHERE valid_until < CURDATE()
    AND is_active = 1;
END;
```

Runs at midnight each night. The seed includes a sample expired code (OLDPROM0, valid_until = '2024-12-31'); after the next event cycle it would flip to is_active = 0.

10. DCL — Roles & Privileges

```
CREATE ROLE IF NOT EXISTS 'driver_role', 'rider_role', 'support_role', 'admin_role';

-- Drivers: read-only on rides
GRANT SELECT ON databaseProject_db.rides TO 'driver_role';

-- Riders: book rides + read their bookings + log payments
GRANT INSERT, SELECT ON databaseProject_db.rides TO 'rider_role';
GRANT INSERT, SELECT ON databaseProject_db.payments TO 'rider_role';
```



```
-- Support team: reads + updates everywhere, BUT no DELETE
GRANT SELECT, UPDATE ON databaseProject_db.* TO 'support_role';
REVOKE DELETE          ON databaseProject_db.* FROM 'support_role';

-- Admin: full power, with grant option
GRANT ALL PRIVILEGES ON databaseProject_db.* TO 'admin_role' WITH GRANT OPTION;

FLUSH PRIVILEGES;
```

10.1 Live SHOW GRANTS evidence

```
mysql> SHOW GRANTS FOR driver_role;
+-----+
| GRANT USAGE ON *.* TO `driver_role`@`%` |
| GRANT SELECT ON `databaseProject_db`.`rides` TO `driver_role` |
+-----+

mysql> SHOW GRANTS FOR rider_role;
+-----+
| GRANT USAGE ON *.* TO `rider_role`@`%` |
| GRANT SELECT, INSERT ON `databaseProject_db`.`payments` TO `rider_role` |
| GRANT SELECT, INSERT ON `databaseProject_db`.`rides` TO `rider_role` |
+-----+

mysql> SHOW GRANTS FOR support_role;
+-----+
| GRANT USAGE ON *.* TO `support_role`@`%` |
| GRANT SELECT, UPDATE ON `databaseProject_db`.* TO `support_role` |
+-----+
(no DELETE ⇒ REVOKE was effective)

mysql> SHOW GRANTS FOR admin_role;
+-----+
| GRANT USAGE ON *.* TO `admin_role`@`%` |
| GRANT ALL PRIVILEGES ON `databaseProject_db`.* TO `admin_role` WITH GRANT OPTION |
+-----+
```

11. Rubric Queries — Backend Reports

11.1 Completed rides for a specific rider, ordered by date

```
SELECT r.ride_id, r.requested_at, r.distance_km, r.duration_min, r.fare,
       du.full_name AS driver_name
FROM rides r
JOIN drivers d ON d.driver_id = r.driver_id
JOIN users du ON du.user_id = d.user_id
WHERE r.rider_id = ? AND r.ride_status = 'COMPLETED'
ORDER BY r.requested_at DESC;
```

Live result for rider_id = 2 (Rider One):

ride_id	requested_at	distance	duration	fare	driver
4	2026-05-05 09:22:51	7.00	15	480.00	Driver Ali

1	2026-05-05 09:22:50	9.50	18	391.50	Driver Ali
---	---------------------	------	----	--------	------------

11.2 Drivers in a city, ordered by rating

```
SELECT DISTINCT d.driver_id, u.full_name, d.avg_rating, d.total_trips, d.avail_status
FROM drivers d
JOIN users u ON u.user_id = d.user_id
JOIN rides r ON r.driver_id = d.driver_id
JOIN locations pl ON pl.location_id = r.pickup_loc_id
WHERE pl.city = ?
ORDER BY d.avg_rating DESC, d.total_trips DESC;
```

Live result for city = "Karachi":

driver_id	full_name	avg_rating	total_trips	avail
3	Driver Bilal	3.90	12	ONLINE
1	Driver Ali	3.00	51	ONLINE

11.3 SUM(amount) — revenue per city

```
SELECT pl.city, COUNT(*) AS paid_rides, ROUND(SUM(p.amount), 2) AS total_revenue
FROM payments p
JOIN rides r ON r.ride_id = p.ride_id
JOIN locations pl ON pl.location_id = r.pickup_loc_id
WHERE p.payment_status = 'PAID'
GROUP BY pl.city
ORDER BY total_revenue DESC;
```

city	paid_rides	total_revenue
Lahore	1	865.00
Karachi	2	784.35

11.4 COUNT(*) — trips per driver

```
SELECT d.driver_id, u.full_name AS driver_name,
COUNT(*) AS completed_trips,
ROUND(SUM(r.fare), 2) AS total_fare_value
FROM rides r
JOIN drivers d ON d.driver_id = r.driver_id
JOIN users u ON u.user_id = d.user_id
WHERE r.ride_status = 'COMPLETED'
GROUP BY d.driver_id, u.full_name
ORDER BY completed_trips DESC;
```

driver_id	driver_name	completed_trips	total_fare_value
1	Driver Ali	2	871.50
2	Driver Sara	1	865.00

11.5 AVG with HAVING < 3.5 — low-rated drivers

```
SELECT u.user_id, u.full_name, d.driver_id,
       ROUND(AVG(rt.score), 2) AS avg_score,
       COUNT(rt.rating_id)    AS rating_count
FROM ratings rt
JOIN users   u ON u.user_id = rt.rated_user
JOIN drivers d ON d.user_id = u.user_id
GROUP BY u.user_id, u.full_name, d.driver_id
HAVING AVG(rt.score) < 3.5
ORDER BY avg_score ASC;
```

user_id	full_name	driver_id	avg_score	rating_count
5	Driver Ali	1	3.00	2

11.6 INNER JOIN — full trip report

```
SELECT r.ride_id, r.ride_status, r.requested_at,
       r.distance_km, r.duration_min, r.fare,
       ru.full_name AS rider_name,
       du.full_name AS driver_name,
       v.make, v.model, v.license_plate, v.vehicle_type,
       pl.city AS pickup_city, dl.city AS dropoff_city
FROM rides r
INNER JOIN users   ru ON ru.user_id = r.rider_id
INNER JOIN drivers d ON d.driver_id = r.driver_id
INNER JOIN users   du ON du.user_id = d.user_id
INNER JOIN vehicles v ON v.vehicle_id = r.vehicle_id
INNER JOIN locations pl ON pl.location_id = r.pickup_loc_id
INNER JOIN locations dl ON dl.location_id = r.dropoff_loc_id
ORDER BY r.requested_at DESC LIMIT 200;
```

ride_id	status	rider	driver	vehicle	plate	type	route	fare
4	COMPLETED	Rider One	Driver Ali	Toyota Corolla	AAA-001	ECONOMY	Karachi → Karachi	480.00
1	COMPLETED	Rider One	Driver Ali	Toyota Corolla	AAA-001	ECONOMY	Karachi → Karachi	391.50
2	COMPLETED	Rider Two	Driver Sara	Honda City	BBB-002	PREMIUM	Lahore → Lahore	865.00
3	IN_PROGRESS	Rider One	Driver Bilal	Suzuki GD110	CCC-003	BIKE	Karachi → Karachi	133.00

11.7 LEFT JOIN — all riders incl. zero-ride riders

```
SELECT u.user_id, u.full_name, u.email,
       COUNT(r.ride_id) AS completed_rides
FROM users u
LEFT JOIN rides r
      ON r.rider_id = u.user_id AND r.ride_status = 'COMPLETED'
WHERE u.role = 'RIDER'
```

```
GROUP BY u.user_id, u.full_name, u.email
ORDER BY completed_rides DESC, u.full_name;
```

user_id	full_name	email	completed_rides
2	Rider One	rider@demo.com	2
3	Rider Two	rider2@demo.com	1
4	Rider Three	rider3@demo.com	0

Note: Rider Three appears with `completed_rides = 0`. An `INNER JOIN` would have dropped this row; the `LEFT JOIN + COUNT(r.ride_id)` explicitly preserves it.

11.8 JOIN payments × promo_codes — discount usage per ride

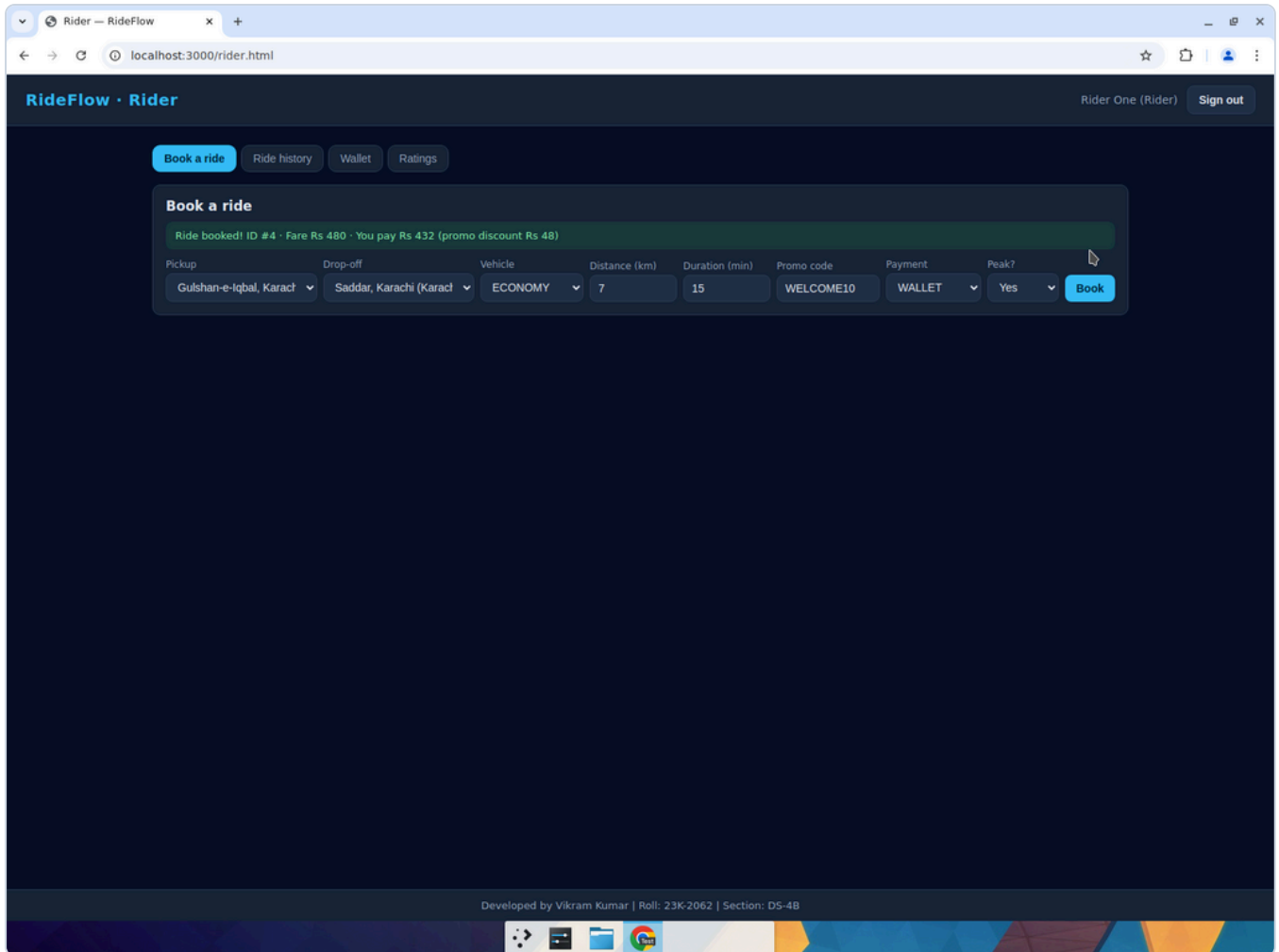
```
SELECT p.payment_id, p.ride_id, pc.code,
       pc.discount_pct, p.promo_discount, p.amount,
       p.payment_status, p.txn_date
FROM   payments p
JOIN   promo_codes pc ON pc.promo_id = p.promo_id
ORDER BY p.txn_date DESC;
```

payment_id	ride_id	code	discount_pct	promo_discount	amount	status
4	4	WELCOME10	10.00	48.00	432.00	PAID
1	1	WELCOME10	10.00	39.15	352.35	PAID

12. UI Evidence — Live End-to-End Flow

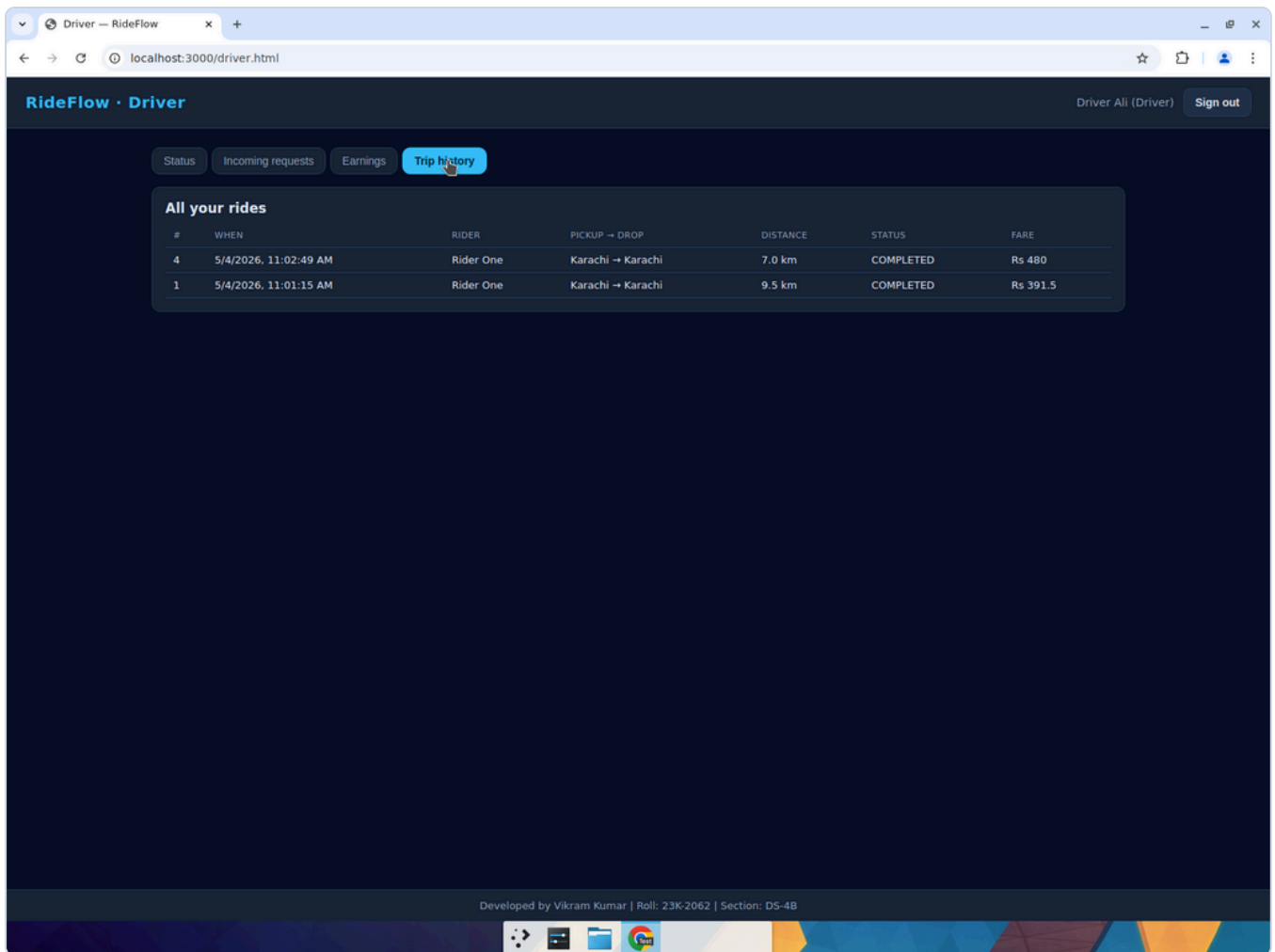
The screenshots below are from a single recorded run: rider books a peak-hour ride with promo → driver completes → rider rates 1★ → admin sees alerts and reports populate. Footer branding "Developed by Vikram Kumar | Roll: 23K-2062 | Section: DS-4B" is visible on every page.

12.1 Booking with stored procedure + promo + surge



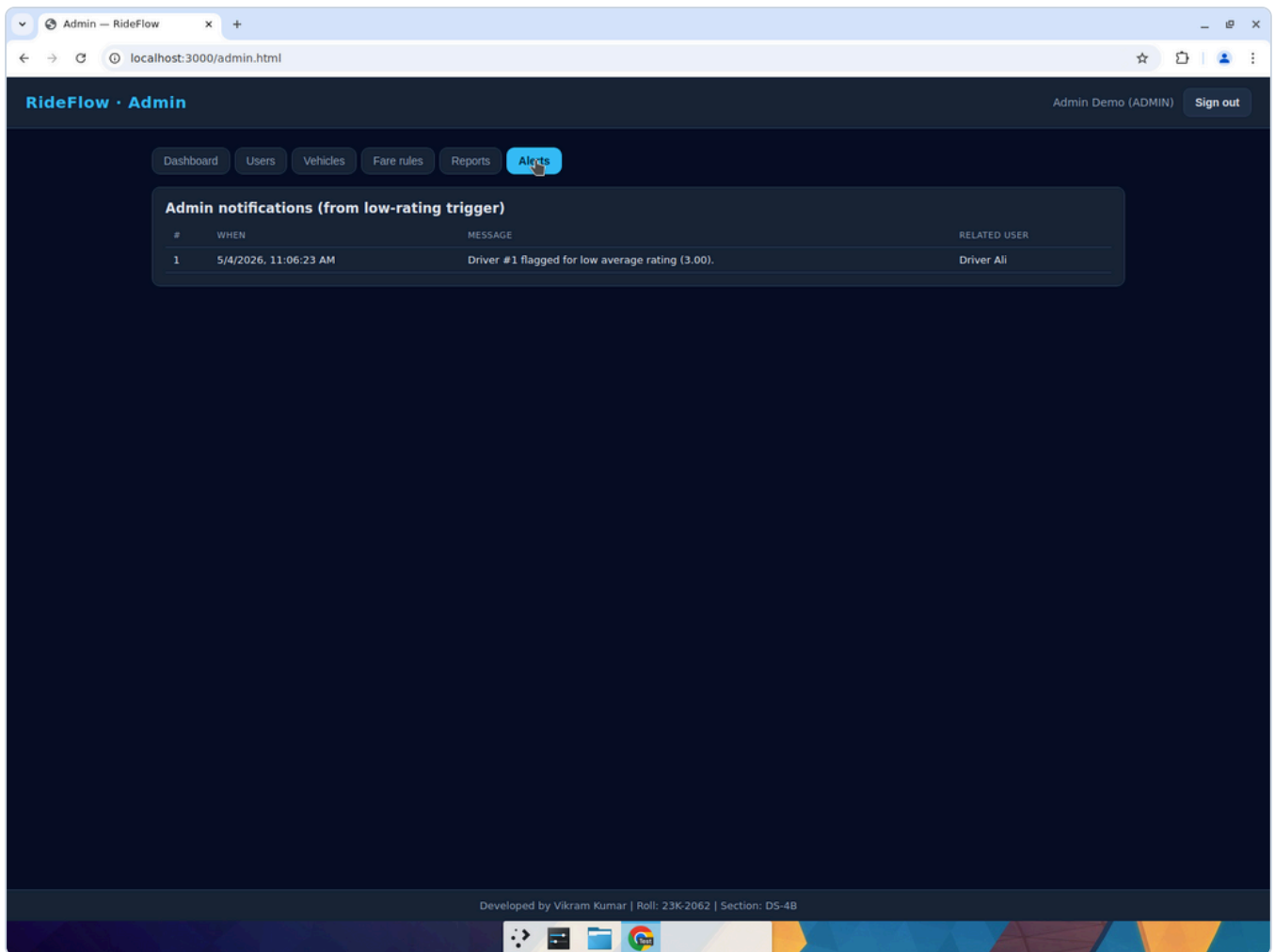
Fare 480 = (100 + 25 × 7 + 3 × 15) × 1.5; promo discount 48; you pay 432.

12.2 Trigger 1 evidence — driver trip history shows COMPLETED



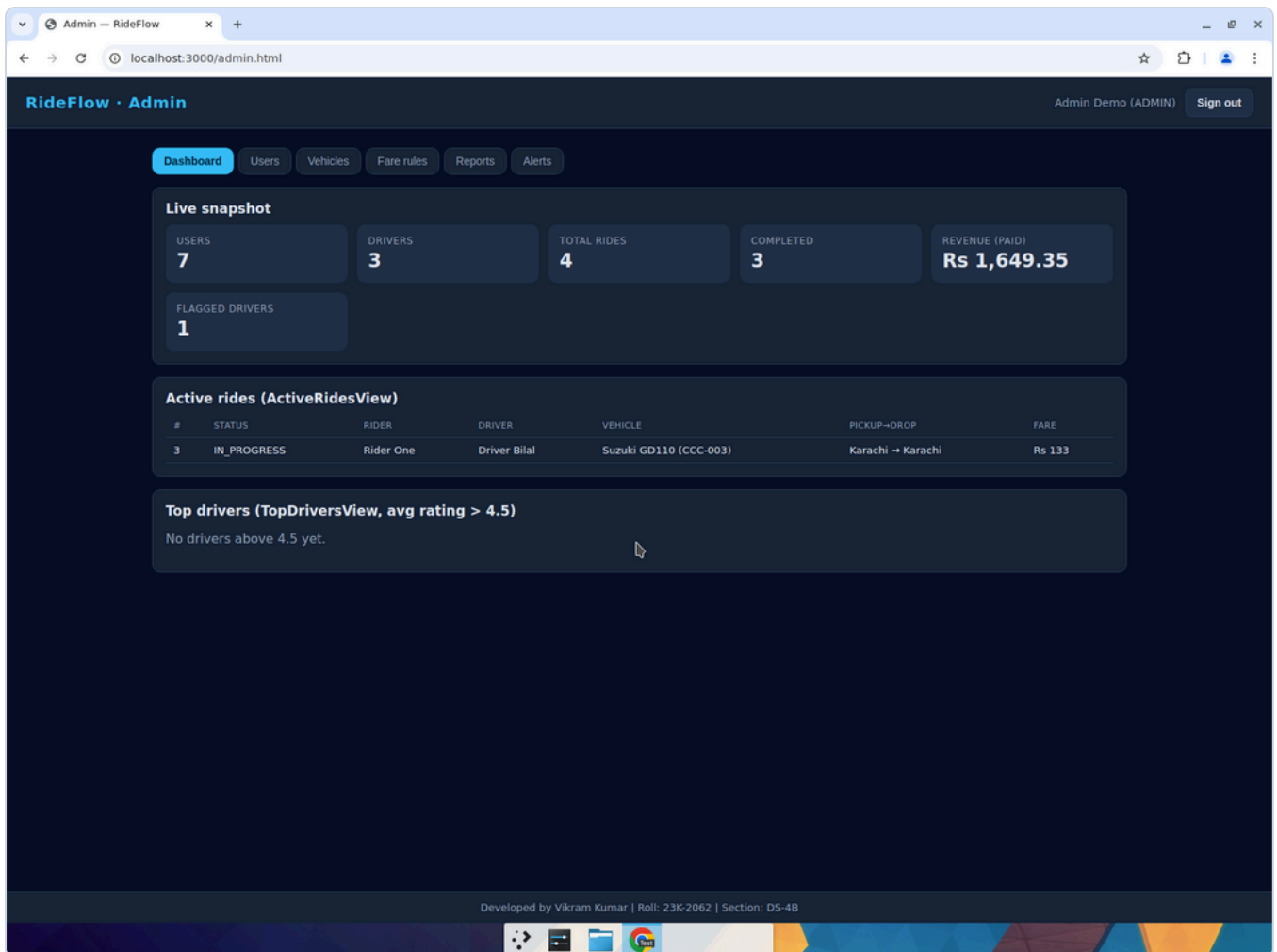
Driver-side /finish only flips payment to PAID; the ride landing in COMPLETED proves *trg_payment_paid_complete_rideran*.

12.3 Trigger 2 evidence — admin notification



1 ★ submitted against Driver Ali → avg dropped to 3.00 → trigger flagged the driver and inserted the alert.

12.4 Admin dashboard — KPIs + ActiveRidesView + TopDriversView



Flagged drivers = 1; ActiveRidesView shows the IN_PROGRESS ride; TopDriversView is empty (no driver above 4.5 after Ali's drop).

12.5 Admin reports — top half (aggregates + INNER + LEFT JOIN)

Admin — RideFlow

localhost:3000/admin.html

RideFlow · Admin

Admin Demo (ADMIN) Sign out

DashboardUsersVehiclesFare rulesReportsAlerts

Revenue per city — SUM(amount) GROUP BY city

CITY	PAID RIDES	TOTAL REVENUE
Lahore	1	Rs 865
Karachi	2	Rs 784.35

Trips per driver — COUNT(*) GROUP BY driver_id

DRIVER	COMPLETED TRIPS	TOTAL FARE
Driver Ali	2	Rs 871.5
Driver Sara	1	Rs 865

Low-rated drivers — HAVING AVG(score) < 3.5

DRIVER	AVG SCORE	RATINGS
Driver Ali	3.00	2

Full trip report — INNER JOIN riders x rides x drivers x vehicles

#	WHEN	RIDER	DRIVER	VEHICLE	PLATE	TYPE	PICKUP—DROP	STATUS	FARE
4	5/4/2026, 11:02:49 AM	Rider One	Driver Ali	Toyota Corolla	AAA-001	ECONOMY	Karachi → Karachi	COMPLETED	Rs 480
1	5/4/2026, 11:01:15 AM	Rider One	Driver Ali	Toyota Corolla	AAA-001	ECONOMY	Karachi → Karachi	COMPLETED	Rs 391.5
2	5/4/2026, 11:01:15 AM	Rider Two	Driver Sara	Honda City	BBB-002	PREMIUM	Lahore → Lahore	COMPLETED	Rs 865
3	5/4/2026, 11:01:15 AM	Rider One	Driver Bilal	Suzuki GD110	CCC-003	BIKE	Karachi → Karachi	IN_PROGRESS	Rs 133

All riders — LEFT JOIN (zero-ride riders included)

#	NAME	EMAIL	COMPLETED RIDES
2	Rider One	rider@demo.com	2
3	Rider Two	rider2@demo.com	1
4	Rider Three	rider3@demo.com	0

12.6 Admin reports — bottom half (promo JOIN + per-rider history)

Admin — RideFlow

localhost:3000/admin.html

Full trip report — INNER JOIN riders x rides x drivers x vehicles

#	WHEN	RIDER	DRIVER	VEHICLE	PLATE	TYPE	PICKUP→DROP	STATUS	FARE
4	5/4/2026, 11:02:49 AM	Rider One	Driver Ali	Toyota Corolla	AAA-001	ECONOMY	Karachi → Karachi	COMPLETED	Rs 480
1	5/4/2026, 11:01:15 AM	Rider One	Driver Ali	Toyota Corolla	AAA-001	ECONOMY	Karachi → Karachi	COMPLETED	Rs 391.5
2	5/4/2026, 11:01:15 AM	Rider Two	Driver Sara	Honda City	BBB-002	PREMIUM	Lahore → Lahore	COMPLETED	Rs 865
3	5/4/2026, 11:01:15 AM	Rider One	Driver Bilal	Suzuki GD110	CCC-003	BIKE	Karachi → Karachi	IN_PROGRESS	Rs 133

All riders — LEFT JOIN (zero-ride riders included)

#	NAME	EMAIL	COMPLETED RIDES
2	Rider One	rider@demo.com	2
3	Rider Two	rider2@demo.com	1
4	Rider Three	rider3@demo.com	0

Promo discount usage — JOIN payments x promo_codes

PAYMENT #	RIDE	PROMO	DISCOUNT %	DISCOUNT RS	NET CHARGED	WHEN
4	4	WELCOME10	10.00	Rs 48	Rs 432	5/4/2026, 11:02:49 AM
1	1	WELCOME10	10.00	Rs 39.15	Rs 352.35	5/4/2026, 11:01:15 AM

Per-rider completed-ride history

Rider

Rider One (#2)

Load

#	WHEN	DRIVER	DISTANCE	FARE
4	5/4/2026, 11:02:49 AM	Driver Ali	7.0 km	Rs 480
1	5/4/2026, 11:01:15 AM	Driver Ali	9.5 km	Rs 391.5

Drivers in city (ordered by rating)

City

Karachi

Load

Developed by Vikram Kumar | Roll: 23K-2062 | Section: DS-4B

12.7 Drivers in city ordered by rating

Admin — RideFlow

localhost:3000/admin.html

Full trip report — INNER JOIN riders x rides x drivers x vehicles

#	WHEN	RIDER	DRIVER	VEHICLE	PLATE	TYPE	PICKUP→DROP	STATUS	FARE
4	5/4/2026, 11:02:49 AM	Rider One	Driver Ali	Toyota Corolla	AAA-001	ECONOMY	Karachi → Karachi	COMPLETED	Rs 480
1	5/4/2026, 11:01:15 AM	Rider One	Driver Ali	Toyota Corolla	AAA-001	ECONOMY	Karachi → Karachi	COMPLETED	Rs 391.5
2	5/4/2026, 11:01:15 AM	Rider Two	Driver Sara	Honda City	BBB-002	PREMIUM	Lahore → Lahore	COMPLETED	Rs 865
3	5/4/2026, 11:01:15 AM	Rider One	Driver Bilal	Suzuki GD110	CCC-003	BIKE	Karachi → Karachi	IN_PROGRESS	Rs 133

All riders — LEFT JOIN (zero-ride riders included)

#	NAME	EMAIL	COMPLETED RIDES
2	Rider One	rider@demo.com	2
3	Rider Two	rider2@demo.com	1
4	Rider Three	rider3@demo.com	0

Promo discount usage — JOIN payments x promo_codes

PAYMENT #	RIDE	PROMO	DISCOUNT %	DISCOUNT RS	NET CHARGED	WHEN
4	4	WELCOME10	10.00	Rs 48	Rs 432	5/4/2026, 11:02:49 AM
1	1	WELCOME10	10.00	Rs 39.15	Rs 352.35	5/4/2026, 11:01:15 AM

Per-rider completed-ride history

Rider

Rider One (#2) Load

#	WHEN	DRIVER	DISTANCE	FARE
4	5/4/2026, 11:02:49 AM	Driver Ali	7.0 km	Rs 480
1	5/4/2026, 11:01:15 AM	Driver Ali	9.5 km	Rs 391.5

Drivers in city (ordered by rating)

City

Karachi Load

DRIVER	AVG RATING	TRIPS	STATUS
Driver Bilal	3.90	12	ONLINE
Driver Ali	3.00	51	ONLINE

13. How to Run

13.1 Local MySQL (full D3 — procedures, triggers, events)

```
git clone https://github.com/Vikram-Kumar-Parmar/RideFlow.git
cd RideFlow
cp .env.example .env                # set DB_TARGET=mysql + MYSQL_*
npm install
npm run db:setup                    # runs schema/01..07 + seed data
npm start                          # http://localhost:3000
```

13.2 TiDB Cloud (+5 bonus mode)

```
cp .env.example .env                # set DB_TARGET=tidb + TIDB_*
npm run db:setup                    # 04..06 (procedures/triggers/events) auto-skipped
npm start
```

The application transparently performs the same logic at the application layer when running on TiDB.

13.3 Demo logins (password is password)

Role	Email
Admin	admin@demo.com
Rider	rider@demo.com (Rider One)
Driver	driver@demo.com (Driver Ali, ECONOMY)

Developed by Vikram Kumar | Roll: 23K-2062 | Section: DS-4B